

Question 1. AdaBoost with Decision Stumps (Classification)

1.1 Methodology

Dataset Preparation: MNIST digits 4 and 9 were loaded and labelled as -1 and $+1$ respectively. From the training set, 1000 samples per class were held out to form the validation set; the remaining samples constitute the training set. PCA was fitted *only* on the training set and used to project both train and validation data to 5 dimensions.

AdaBoost Algorithm: At each boosting round $t = 1, \dots, 300$:

1. Grow a decision stump $h_t(x)$ on the *weighted* training set. For each of the 5 dimensions, candidate split thresholds are mid-points between consecutive unique sorted values; up to 1000 mid-points per dimension are sampled for speed.
2. Compute weighted misclassification error: $\varepsilon_t = \sum_i w_i \cdot \mathbf{1}[h_t(x_i) \neq y_i]$
3. Compute the tree weight: $\alpha_t = \frac{1}{2} \ln\left(\frac{1-\varepsilon_t}{\varepsilon_t}\right)$
4. Update and re-normalise sample weights: $w_i \leftarrow w_i \exp(-\alpha_t y_i h_t(x_i))$

Validation accuracy is recorded after every boosting round and the iteration that yields the highest validation accuracy is used to evaluate the test set.

1.2 Results

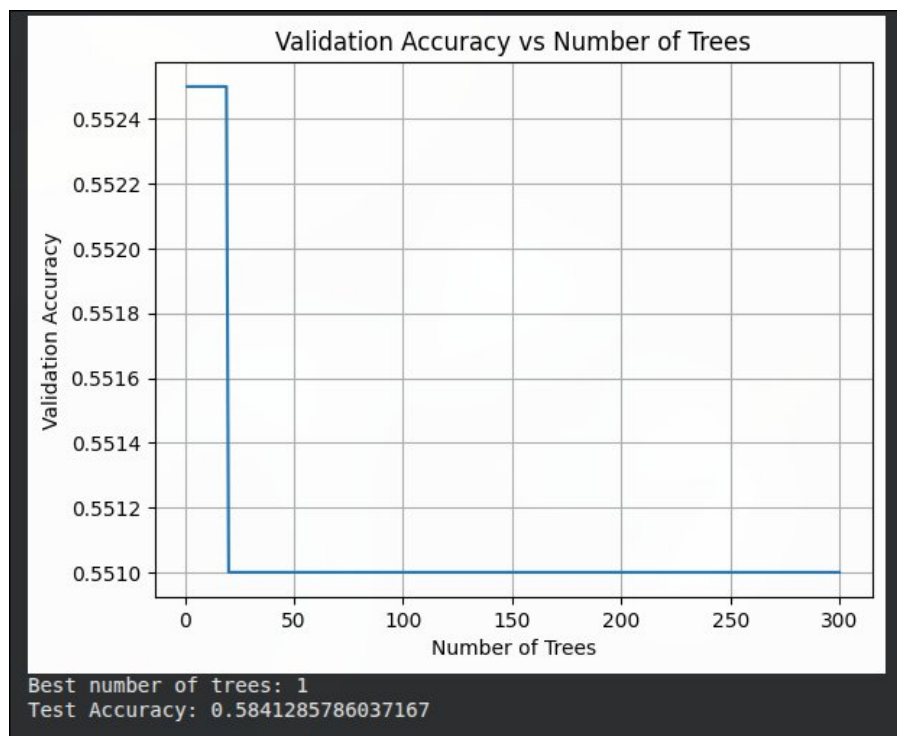


Figure 1: Validation accuracy vs. number of AdaBoost trees (stumps).

- **Best number of trees:** 1
- **Test Accuracy:** ≈ 0.5841 (58.41%)

1.3 Discussion

The validation accuracy peaks immediately at $t = 1$ (single stump) and then sharply drops before plateauing around 0.551. This indicates that a single weighted stump is already picking up the most discriminative direction in the 5-D PCA space, but subsequent stumps (which focus on the residual hard examples) do not generalise — likely because the 5-dimensional projection discards too much class-discriminative information for further refinement via boosting. The test accuracy of $\approx 58\%$ is only slightly above chance for a two-class problem, confirming that 5 PCA components are insufficient for digits 4 vs. 9 without deeper models.

Question 2. Gradient Boosting with Decision Stumps (Regression/L2)

2.1 Methodology

Dataset Preparation: Same PCA pre-processing as Question 1.

Gradient Boosting for Absolute Loss (via residuals): At each iteration t :

1. Fit stump $h_t(x)$ to minimise SSR on the current pseudo-residuals (negative gradients of the loss).
2. Update the prediction: $F_t(x) = F_{t-1}(x) + \eta \cdot h_t(x)$
3. Compute new pseudo-residuals: $r_i = y_i - F_t(x_i)$

Validation MSE is recorded after every iteration; the iteration with the lowest validation MSE determines the final model.

2.2 Results

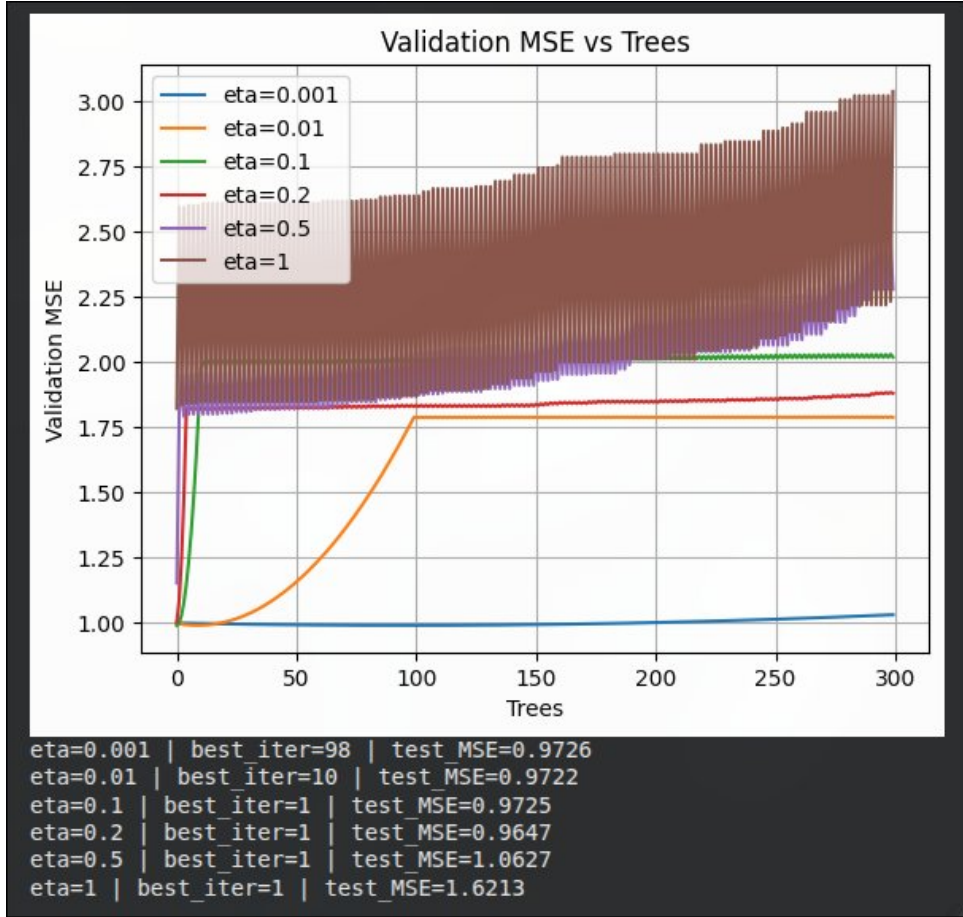


Figure 2: Validation MSE vs. number of gradient-boosting trees for various learning rates η .

Table 1: Summary of results across learning rates.

η	Best Iteration	Test MSE
0.001	98	0.9726
0.01	10	0.9722
0.1	1	0.9725
0.2	1	0.9647
0.5	1	1.0627
1.0	1	1.6213

2.3 Discussion

Trends and patterns:

- **Small η (0.001):** The model learns slowly; validation MSE keeps decreasing well beyond 10 iterations before the best point is reached at iteration 98. Training MSE also descends slowly and the gap between train and validation MSE remains small — low variance, but slow convergence.

- **Moderate η (0.01–0.1):** The best performance is achieved with moderate learning rates. $\eta = 0.2$ gives the lowest test MSE of 0.9647. A few iterations suffice and the model avoids both slow convergence and overshoot.
- **Large η (≥ 0.5):** The model immediately overshoots; validation MSE rises steeply from iteration 1, and the best iteration is trivially 1. The test MSE degrades significantly (> 1.0). This is consistent with the gradient step being too large, causing oscillation around the minimum.
- **General pattern:** Smaller learning rates require more trees to reach the optimal solution but yield more stable training curves. Larger learning rates converge in fewer trees but risk overshooting and diverging. The sweet spot ($\eta \approx 0.1$ – 0.2) balances speed and stability.

Question 3. Rosenblatt Perceptron

3.1 Methodology

Dataset Generation:

- **Dataset A:** Class -1 : $X \sim \mathcal{N}\left(\begin{bmatrix} -3 \\ -3 \end{bmatrix}, I\right)$, Class $+1$: $X \sim \mathcal{N}\left(\begin{bmatrix} 3 \\ 3 \end{bmatrix}, I\right)$; 200 samples per class.
- **Dataset B:** Same means, covariance $3I$; 200 samples per class.

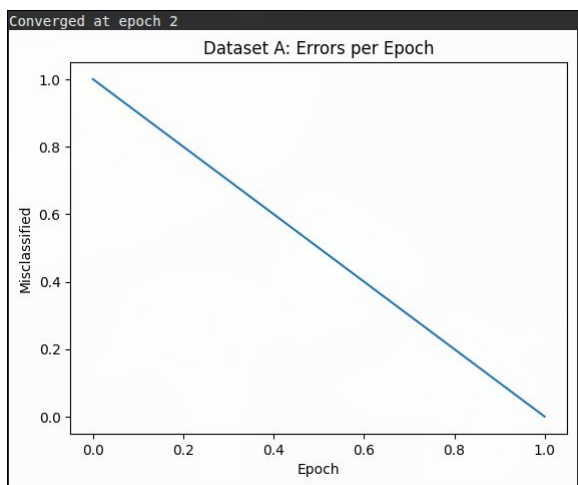
Both datasets use a 70-30 train-test split. All weights and biases are initialised to zero.

Perceptron Update Rule (implemented from scratch): For each misclassified sample (x_i, y_i) :

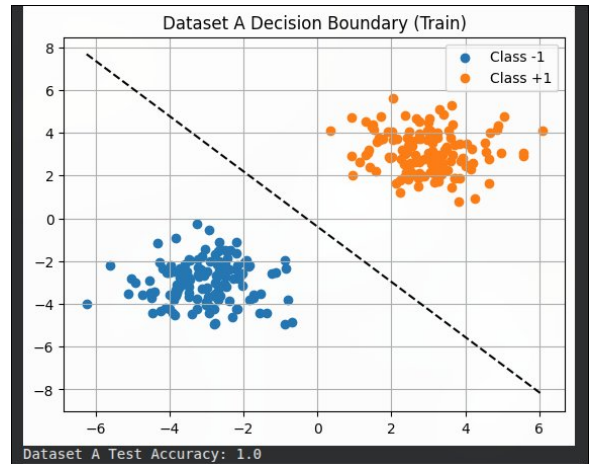
$$w \leftarrow w + \eta y_i x_i, \quad b \leftarrow b + \eta y_i$$

with $\eta = 0.01$, up to 300 epochs. Early stopping is triggered if an entire epoch produces zero misclassifications.

3.2 Dataset A — Linearly Separable



(a) Misclassified samples per epoch.



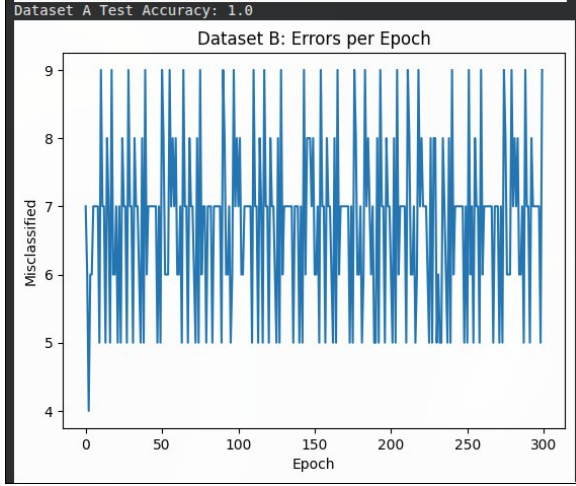
(b) Learned decision boundary on training data.

Figure 3: Rosenblatt Perceptron — Dataset A.

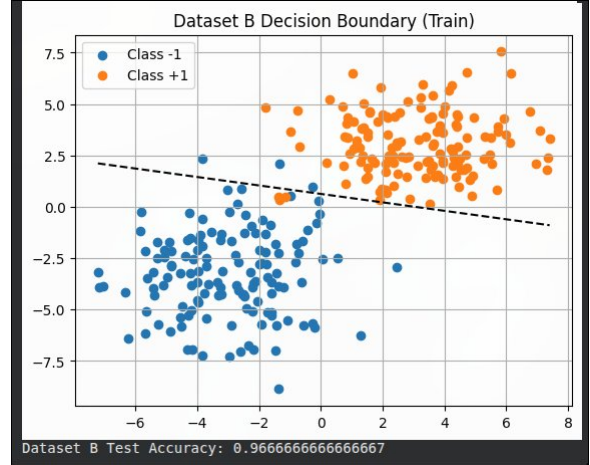
- **Convergence epoch:** 2
- **Test Accuracy:** 1.0 (100%)

The perceptron converges in just 2 epochs because Dataset A is *perfectly linearly separable* (unit covariance, well-separated means). Misclassifications drop linearly from epoch 0 to 1, hitting zero at epoch 2. The decision boundary cleanly separates the two Gaussian clusters.

3.3 Dataset B — Overlapping Classes



(a) Misclassified samples per epoch (no convergence).



(b) Learned decision boundary on training data.

Figure 4: Rosenblatt Perceptron — Dataset B.

- **Convergence:** Not observed (training ran for all 300 epochs)
- **Test Accuracy:** ≈ 0.967 (96.7%)

3.4 Discussion & Justification

- **Dataset A converges rapidly** because the perceptron convergence theorem guarantees convergence in finite steps for linearly separable data. With large margin (distance between cluster centres = $\|\Delta\mu\| = 6\sqrt{2}$) and tight clusters (covariance I), very few weight updates are needed.
- **Dataset B never converges** because the larger covariance ($3I$) causes the two Gaussian clusters to *overlap* near the boundary. Overlapping data is **not linearly separable**, so no single hyperplane can perfectly classify all training points, and the perceptron oscillates indefinitely, trying to correct each newly misclassified overlap sample.
- **Error trajectory in Dataset B** oscillates chaotically between roughly 4–9 misclassifications across all 300 epochs, confirming that the weights cycle without settling.
- **Test accuracy still at 96.7%** for Dataset B: even though convergence is not achieved, the perceptron has learned a reasonable boundary. Since most of the

class mass is still well-separated (overlap is only in the tails), the learned hyperplane correctly classifies the majority of test points.

- **Key takeaway:** The Rosenblatt perceptron is guaranteed to converge *only* for linearly separable datasets. When classes overlap, one should instead use soft-margin SVMs, logistic regression, or other models that tolerate non-separability.